

那么，`pthread`s 该如何支持任意类型的返回值呢？注意到 `pthread`s 提供的 POSIX 接口并不是系统调用，而是在用户态运行的线程库函数。因此，`pthread`s 可以在用户态设计相应的数据结构，用于保存线程的返回值。

如图 6.12 所示，`pthread`s 与线程相关的数据结构实际上被拆分成两部分：内核结构（`tcb`）依然保存与线程相关的重要信息（如所属进程和处理器上下文），这些信息不可被用户直接访问；其他信息则保存在用户态的数据结构（`pthread`）中，内核并不知晓这些数据与该线程的关系。

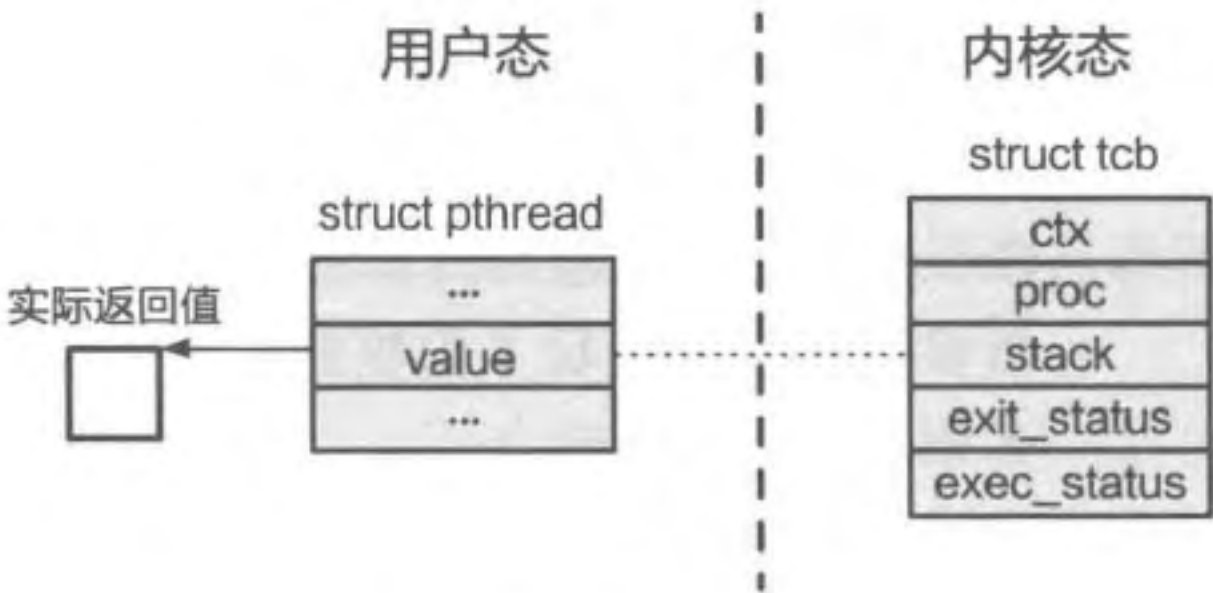


图 6.12 pthreads 与线程相关的数据结构分为两部分

由于线程在内核态和用户态的执行实际上拥有较强的独立性，我们可以将其视为两类线程。内核态线程（Kernel Level Thread）是由内核创建并直接管理的线程，内核维护了与之对应的数据结构——TCB。而用户态线程（User Level Thread）是由用户态的线程库创建并管理的线程，其对应的数据结构保存在用户态，内核并不知晓该线程的存在，也不对其进行管理。由于内核只能管理内核态线程，其调度器（第 7 章会详细介绍）只能对内核态线程进行调度，因此用户态线程如果想要执行，就需要“绑定”到相应的内核态线程才能执行。

读者可能会对这“两类线程”感到疑惑：根据第 3 章的介绍，`pthread`s 作为线程库只是提供了更加方便的抽象，其背后还是对应着由内核管理的线程，为什么要将其拆分为内核态线程和用户态线程呢？这个问题在 `pthread`s 场景下确实不好回答，因为它的用户态线程与内核态线程始终是一一对应的。但在本节中读者将发现，用户态线程与内核态线程的关系还可以更加复杂，而刻画用户态线程与内核态线程关系的方法就称为多线程模型（Multithreading Model）。一般来说，多线程模型分为以下三类（如图 6.13 所示）。

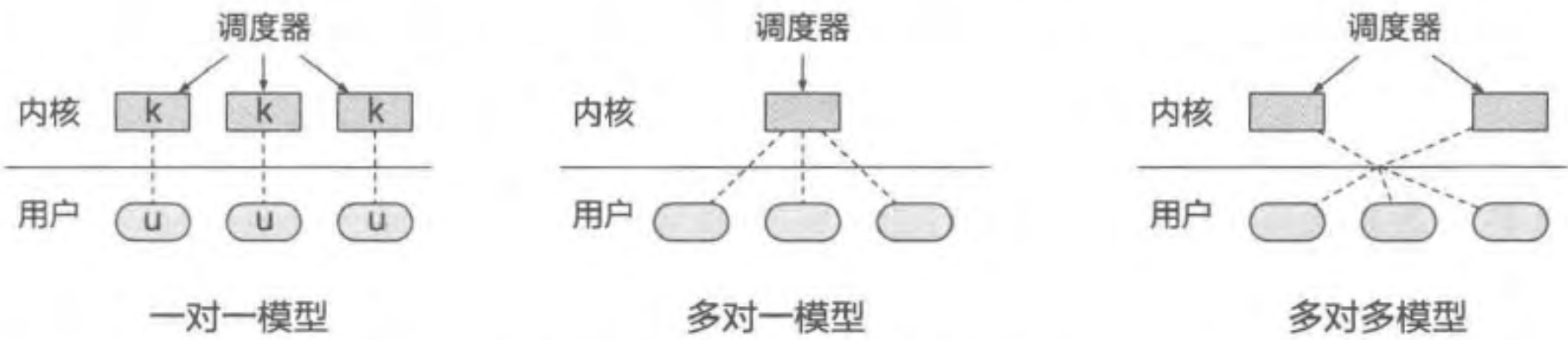


图 6.13 三种不同的多线程模型

- 一对一模型。一对一模型是现在最为常见的多线程模型，在这种模型中，每个用户态线程都对应一个单独的内核态线程。因此，当内核态线程被内核调度执行时，其对应的用户态线程就会被执行，两者关系非常紧密，看起来就像是同一个线程

的两个部分。pthreads 采取的就是这种一对一模型。

- 多对一模型。多对一模型允许将多个用户态线程映射到单一的内核态线程。如图 6.13 所示，在这种模型下，进程实际上只分配一个内核态线程，其多线程环境完全由用户态的线程库实现。比较有名的使用多对一模型的线程库是 Green Thread，它最初是由 Sun 公司为其 Java 应用开发的。由于当时的计算机资源（如内存）有限，创建内核态线程会给系统带来较大的压力。与内核态线程相比，Green Thread 创建的线程状态及其执行情况完全由用户态线程库管理，不需要内核参与，从而减轻了内核的压力。Green Thread 一词后来也成为用户态线程的代名词，可见其影响力。
- 多对多模型（ $M:N$ ）。多对一模型有一个显著的问题：由于进程中只包含一个内核态线程，因此同时只能被调度到一个 CPU 上执行，这导致应用的可扩展性受限。针对这一问题，Sun 公司又提出了多对多模型，它允许应用自定义进程中包含的内核态线程数量（例如可以和 CPU 数量保持一致），之后由线程库将用户态线程映射到不同的内核态线程执行。这种模型既突破了多对一模型的可扩展性限制，又减少了一对一模型存在的资源开销。

总的来说，多对多模型和多对一模型最初都是针对资源受限场景提出的，可以在占用较少内核资源的情况下创建大量线程。除此之外，用户态线程的创建、切换、销毁等功能都不需要进入内核，因此时延较低。但是，用户态线程的大量引入也会让线程管理变得更加复杂。随着计算机资源的丰富及 pthreads 的普及，多对多和多对一模型逐渐被更加简洁的一对一模型取代，就连 Sun 公司的 Solaris 操作系统也在第 9 版之后改为了一对一模型。不过，多对多 / 多对一模型并没有完全消亡，近年来随着纤程的发展，它们又迸发了新的活力。

6.6 纤程

本节主要知识点

- 为什么对纤程的需求在近年来越发强烈？
- 什么是纤程？如何使用纤程？
- 纤程如何进行切换？
- 纤程与进程、线程的区别有哪些？

以 pthreads 为代表的一对一线程模型在用户态线程和内核态线程之间建立了紧密的联系，用户态线程可以利用内核支持完成多种功能（如第 9 章将介绍的线程同步），因而得到了比较广泛的使用。但是，随着计算机的发展，应用类型和需求渐趋多样化，一对

一模型的劣势也逐渐凸显出来。首先，比较复杂的应用可能会包含大量线程，且每个线程各司其职，有的负责计算，有的负责网络通信，有的负责大量读写内存。与操作系统调度器相比，应用对线程的语义和执行状态更加了解，因此可能做出更优的调度决策。其次，一对一模型的线程创建对应着内核态线程的创建，其创建时间较长，对于执行时间较短（如微秒级）的任务时延会造成较大影响。最后，一对一模型中线程的切换涉及内核态线程切换，因此必须进入内核，其性能开销也相对较高，对一些交互性强的线程（如负责网络收发的线程）影响较大。在这样的背景下，人们再次将目光转向了用户态线程，并提供了更多对于纤程（Fiber）的支持。

纤程、协程与用户态线程

相比纤程，熟悉高级语言的读者可能更经常接触到另一个概念——协程（Coroutine），它为应用提供了轻量级的用户态可并行单元。实际上，协程与纤程没有太大区别，只是所处的语境不同。纤程一般用来描述操作系统提供的用户态可并行单元支持（系统概念），而协程则用来描述程序语言提供的可并行抽象（语言概念）。不过有的时候这两个名称也会混用（例如为 JavaScript 提供的一种协程支持库就叫 node-fibers）。

那么，纤程/协程与用户态线程的关系是什么呢？显然，纤程/协程符合用户态线程的定义，因为它们由用户态的库函数创建和管理，内核并不知晓它们的存在。我们可以认为纤程和协程是用户态线程的一种，因为它们有明确的调度方法（之后会详细介绍），而用户态线程的定义中并未明确说明如何进行调度。不过，现有纤程和协程通常比 6.5.6 节介绍的用户态线程抽象层次更高，因为它们往往是在现有线程库的基础上再实现的抽象。

举例来说，现有的纤程/协程库通常依赖于 pthreads，可以直接利用 pthreads 提供的丰富功能。而过去的用户态线程库（如 Green Thread）直接与内核态线程对接，因此还需要对内核态线程进行管理（如直接使用系统调用创建内核态线程），实现较为复杂。因此，我们可以将纤程和协程简单理解为一种轻量级的用户态线程实现。

6.6.1 对纤程的需求：一个简单的例子

本节将首先引入一个简单的例子来更加深入地分析纤程相比线程的优势。这个例子被称为生产者-消费者模型，它是计算机领域常用的经典模型。在接下来的章节中，我们还会经常使用这一模型介绍操作系统中的各种概念和机制。顾名思义，生产者-消费者模型包含两个部分：生产者和消费者。其中，生产者负责“生产”数据，即生成一些用于处理的数据；消费者则负责“消费”数据，即处理生产者生成的数据。

现在，假设一个进程拥有两个线程，一个线程是生产者，另一个线程是消费者。由于两个线程共享同一地址空间，生产者可以在共享的内存里直接写入数据，供消费者使

用。假设该计算机只有一个处理器，那么数据从生产者生成到消费者处理至少需要经历一次线程切换（即从生产者切换到消费者）。而在实际情况中，由于该计算机上可能运行了很多程序，而调度器并不知道生产者与消费者之间的关系，因此未必会优先选择消费者进行调度。因此，尽管生产者早已完成了数据的生成，但由于线程切换的开销和调度器的选择，消费者可能需要经历较长时间的延迟才能开始处理数据。为了消除这部分延迟，应用可以使用纤程。下面，本节将以 POSIX 提供的纤程支持 `ucontext` 为例具体分析纤程带来的好处。

6.6.2 POSIX 的纤程支持：ucontext

POSIX 用来支持纤程的是 `ucontext.h` 中的接口：

```

1 #include <ucontext.h>
2
3 int getcontext(ucontext_t *ucp);
4 int setcontext(const ucontext_t *ucp);
5 void makecontext(ucontext_t *ucp, void (*func)(), int argc, ...);
6

```

其中，`getcontext` 用来保存当前的纤程上下文（主要包括栈和处理器上下文），而 `setcontext` 则用来切换到另一个纤程上下文执行。最后，`makecontext` 会修改纤程上下文，使其从指定的地址开始执行。`makecontext` 接收的参数包括纤程上下文的起始执行地址（`func`）、执行参数数量（`argc`）及参数列表。由于这些上下文属于纤程，因此并不会保存到内核的 TCB 中，而是完全保存在用户态。代码片段 6.39 展示了如何使用这些接口实现一个简单的生产者 - 消费者模型（略去了部分用于错误处理的分支）。

代码片段 6.39 通过 POSIX 的 `ucontext` 系列接口实现的生产者 - 消费者模型

```

1 #include <signal.h>
2 #include <stdio.h>
3 #include <ucontext.h>
4 #include <unistd.h>
5
6 ucontext_t ucontext1, ucontext2;
7
8 int current = 0;
9
10 void produce()
11 {
12     current++;
13     // 切换到消费者纤程执行
14     setcontext(&ucontext2);
15 }
16

```

```

17 void consume()
18 {
19     printf("current value: %d\n", current);
20     // 切换到生产者线程执行
21     setcontext(&ucontext1);
22 }
23
24 int main(int argc, const char *argv[])
25 {
26
27     char iterator_stack1[SIGSTKSZ];
28     char iterator_stack2[SIGSTKSZ];
29
30     // 初始化生产者线程
31     getcontext(&ucontext1);
32     ucontext1.uc_link = NULL;
33     ucontext1.uc_stack.ss_sp = iterator_stack1;
34     ucontext1.uc_stack.ss_size = sizeof(iterator_stack1);
35     makecontext(&ucontext1, (void (*)(void))produce, 0);
36
37     // 初始化消费者线程
38     getcontext(&ucontext2);
39     ucontext2.uc_link = NULL;
40     ucontext2.uc_stack.ss_sp = iterator_stack2;
41     ucontext2.uc_stack.ss_size = sizeof(iterator_stack2);
42     makecontext(&ucontext2, (void (*)(void))consume, 0);
43
44     // 切换到生产者线程执行
45     setcontext(&ucontext1);
46
47     return 0;
48 }

```

在主函数中，该程序使用 `makecontext` 创建了两个上下文 `ucontext1` 和 `ucontext2`，分别调用 `produce` 和 `consume` 函数。需要注意的是，这两个上下文的栈需要由应用程序准备。之后，应用调用 `setcontext`，进入 `ucontext1` 对应的 `produce` 函数执行。此后，该应用通过 `setcontext` 在 `produce` 和 `consume` 之间跳转，不断重复“生产”和“消费”的过程。图 6.14 展示了该程序中控制流的变化示意图。

代码片段 6.39 说明，通过利用纤程，可以对生产者-消费者这类需要多个模块协作的场景进行有效支持。当生产者完成任务后，可以立即切换到消费者继续执行。由于该切换是通过用户态线程库完成的，不需要内核的参与，也不受内核调度器的控制，因此可以达到很好的性能。下面将

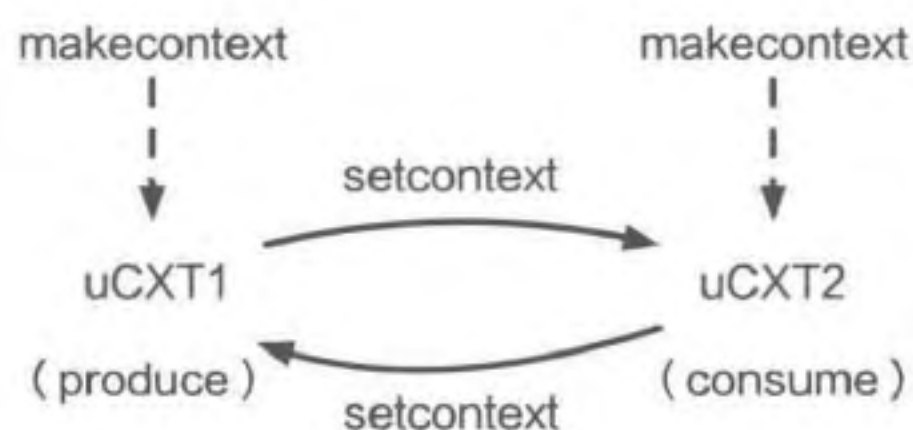


图 6.14 代码片段 6.39 中的控制流变化示意图

对线程切换进行更加详细的介绍。

6.6.3 线程切换

线程切换的触发机制与主流一对一模型的线程存在一定差别。在前面的进程切换部分提到，操作系统可以通过中断的方式使进程进入内核态，从而完成切换。这种“被动切换”是强制的，基于这种切换实现的协作方式称为抢占式多任务处理（Preemptive Multitasking）模式。而线程并不具备使用中断抢占其他线程的权限，因此通常无法使用这种模式。所以，线程库一般会提供 `yield` 接口，使线程主动调用该接口并切换到其他线程执行。基于这种切换实现的协作方式需要多个线程进行协作以完成线程调度，被称为合作式多任务处理（Cooperative Multitasking）模式。在 `ucontext` 中，`setcontext` 提供了使线程放弃 CPU 资源、切换到其他线程的功能。

代码片段 6.40 展示了 `glibc` 在 AArch64 架构下对于 `setcontext` 的实现。可以看出，这部分代码与 ChCore 进程 / 线程切换中的 `exception_exit`（代码片段 6.27）比较相似，都会完成恢复寄存器、换栈、切换 PC 等操作。不过，由于 `setcontext` 是在用户态完成切换，不涉及内核态和用户态之间的切换，也不涉及对于前一个处理器上下文的保存，因此其性能明显优于线程切换。另外，`setcontext` 只恢复被调用者保存的通用寄存器，而 `exception_exit` 除恢复通用寄存器以外，还需要恢复部分特殊寄存器和系统寄存器。最后，由于 `setcontext` 是在用户态完成切换，因此是通过间接跳转指令 `br` 完成 PC 的切换；而 `exception_exit` 由于涉及内核态到用户态的切换，因此使用了特殊指令 `eret`。这体现出线程在用户态切换的两点优势：第一，保存的状态较少；第二，不需要引入内核态和用户态之间的切换。经过测试，在使 AArch64 架构的华为鲲鹏 916 服务器上：如果使用线程，那么从生产者线程切换到消费者线程需要约 1900 纳秒；而如果使用线程，该切换时间将减少到约 500 纳秒。从中可以看出线程切换在性能上的优势。

代码片段 6.40 AArch64 上 `setcontext` 的代码片段

```
1 ENTRY (__setcontext)
2   ...
3   // 恢复被调用者保存的通用寄存器
4   ldp x18, x19, [x0, register_offset + 18 * SZREG]
5   ldp x20, x21, [x0, register_offset + 20 * SZREG]
6   ldp x22, x23, [x0, register_offset + 22 * SZREG]
7   ldp x24, x25, [x0, register_offset + 24 * SZREG]
8   ...
9   // 恢复用户栈
10  ldr x2, [x0, sp_offset]
11  mov sp, x2
12  // 恢复浮点寄存器及参数
13  ...
```



```

14 // 恢复 PC 并返回
15 ldr x16, [x0, pc_offset]
16 br x16

```

练习

- 6.10 在编写 WordCount 程序（统计文章中每个单词的出现次数）时，小明决定使用并发编程，将统计过程划分为多个不相干的子任务。现在小明有三个选项可实现并发：创建多个进程、线程或者纤程。请问哪个选项更加合理？请从性能和多核并发性两个角度阐述原因。

6.7 思考题

1. 假设在 Linux 中，一个进程在打开一个文件以后调用 fork 创建了一个子进程。此时，该子进程能直接对该文件进行读取吗？分析一下 Linux 是怎样实现 fork 来达到这个目标的。
2. 为什么在 fork 的实现中不需要拷贝内核栈？
3. 假设小明实现了 `int thread_create(void (* fp)(int *), int *data)` 和 `void join(void)` 两个线程相关的函数，其中 `thread_create` 能创建一个从函数 `fp` 开始执行并使用 `data` 作为参数的线程，而 `join` 会一直等待直到所有线程运行结束。考虑下面的斐波那契数列计算程序：

```

1 int fib(int n)
2 {
3     if(n <= 1)
4         return 1;
5     else
6         return fib(n-1) + fib(n-2);
7 }

```

- (a) 该程序为单线程版本，请使用上面的两个接口帮助小明实现一个多线程版本。
 - (b) 假设小明在一台拥有两个物理核心的机器上运行该程序（即最多可以同时运行两个线程），发现该程序的性能还不如单线程版本。请帮助他分析原因。
4. 参考 `vfork`、`clone`、`posix_spawn` 的设计，为 `fork` 提出针对大内存应用的优化策略。要求：`fork` 必须同时支持“`fork+exec`”和“`fork`后直接使用”两种场景。
 5. 纤程一般采用合作多任务处理模式执行。如果让你为纤程添加抢占式的调度支持，你会如何实现呢？

6.8 练习答案

- 6.1 第二个进程调用 `process_create` 创建第一个进程；第二个进程调用 `process_waitpid` 等待第一个进程退出；第一个进程接收到用户输入的数字后，使用 `process_exit` 退出，并将数字作为返回值；第二个进程从 `process_waitpid` 调用返回并获取 `process_exit` 的返回值，然后输出到屏幕上。
- 6.2 最简单的一种：创建→就绪→运行→僵尸→退出。根据 `grep` 的实现，中间还可能存在阻塞的情况，以及多次在就绪、运行、阻塞间切换的情况。
- 6.3 $N=2$ 时的输出为三行 `hello`， $N=4$ 时的输出为八行 `hello`。
- 6.4 分别是：`fork`、`posix_spawn`、`fork`。这是因为 `fork` 创建的进程之间联系更加紧密，拥有相似的状态，也容易设置共享内存，而 `posix_spawn` 则适合创建全新进程的场景，其性能一般会优于 `fork`。
- 6.5 这是因为进程在内核中的栈帧状态可能也需要保存。考虑下面的场景，如果进程发生缺页异常，且操作系统发现该物理页已经被换出到硬盘，需要重新加载。由于加载过程较慢，操作系统可能会切换到别的进程执行，此时原进程在内核中的栈帧需要保存，否则之后无法正确恢复执行。为了保证这种场景的正确性，操作系统为每个进程单独维护内核栈，并保存它们进入内核后执行的栈帧信息。
- 6.6 `switch_vmspace` 中的虚拟地址空间切换不再需要了，但依然需要把内核栈顶地址调整到处理器上下文存储的位置，方便进程恢复处理器上下文和返回用户态。
- 6.7 这是因为 `ChCore` 的内核栈地址可以从处理器上下文所在地址推导出来。对于内核栈底地址，由于处理器上下文所在地址固定在栈底，所以 `PCB` 中只要维护指向处理器上下文的指针即可。而对于内核栈顶地址，由于 `ChCore` 在内核栈切换时一定会舍弃其他栈帧并直接切换到处理器上下文的起始地址，因此也没必要单独保存。
- 6.8 `cap_group` 里只需要保存 `slot_table` 即可，但其中的 `slot` 需要保存进程的所有线程。用户态的 `proc` 结构体则要把 `exit_status` 和 `exec_status` 移到线程相关的结构体，并加入子进程列表和线程总数相关状态。
- 6.9 正文中已经提示过：进程包含的内容多且有一定的相似性，因此用拷贝的方法创建比较方便；而线程包含的内容较少，且通常在运行时各不相同，因此直接创建更合适。
- 6.10 线程最为合理。从性能的角度看，线程具有最优的创建和切换性能，线程次之，进程最差；从多核并发性的角度看，线程需要依托线程才能实现在多个核心上运行，而多线程和多进程都能直接利用多核资源。综合来看选择线程。

参考文献

- [1] NYMAN L, LAAKSO M. Notes on the history of fork and join [J]. IEEE Annals of the History of Computing, 2016, 38 (3): 84-87.
- [2] RITCHIE D M, THOMPSON K. The unix time-sharing system [J]. Bell System Technical Journal, 1978, 57 (6): 1905-1929.
- [3] RITCHIE D M. The evolution of the unix time-sharing system [C] // Symposium on Language Design and Programming Methodology. 1979: 25-35.
- [4] BAUMANN A, APPAVOO J, KRIEGER O, et al. A fork() in the road [C] // Proceedings of the Workshop on Hot Topics in Operating Systems. 2019: 14-22.

进程与线程：扫码反馈

